Task 2

Sensor-Based IoT Simulation

LED Control using Temperature • Light • Motion Sensors

Tinkercad / Proteus — No Hardware Required

Arduino UNO 3 Sensors 3 LEDs IoT System No Hardware

```
Arduino UNO  |  TMP36 Temperature  |  LDR Light Sensor  |  PIR HC-SR501 Motion
```

Project Summary	
Platform	Tinkercad (online circuit simulator) / Proteus
Microcontroller	Arduino UNO (ATmega328P, 5V, 16MHz)
Sensors	TMP36 (Temperature) + LDR (Light) + PIR HC-SR501 (Motion)
Actuators	3x LEDs (Red, Blue, Yellow) with 220Ω current-limiting resistors
Language	Arduino C++ (compiled with avr-gcc)
Serial Monitor	9600 baud — sensor readings output every 500 ms

1. Project Overview

This project simulates an IoT (Internet of Things) system using an Arduino UNO microcontroller connected to three types of sensors: a TMP36 temperature sensor, an LDR (Light Dependent Resistor) for ambient light detection, and a PIR (Passive Infrared) sensor for motion detection. Each sensor controls a dedicated LED output, forming a complete sensor-actuator IoT loop — all simulated in Tinkercad without requiring any physical hardware.

The system demonstrates three fundamental IoT patterns: threshold-based alerts (temperature), adaptive automation (light-controlled LEDs), and event-driven responses (motion detection). All sensor readings are broadcast via the Arduino's Serial Monitor at 9600 baud for real-time monitoring and debugging.

2. Circuit Diagram & Wiring

The schematic below shows the complete wiring. Sensors connect to Arduino input pins; LEDs (with current-limiting resistors) connect to output pins.

Schematic Diagram:

```
ARDUINO UNO +-------------------+ [TMP36] ----> | A0 D9 |---> 220ohm ---> [RED LED] ---> GND
[LDR+10k] --> | A1 D10 |---> 220ohm ---> [BLUE LED] ---> GND [PIR] ----> | D2 D11 |---> 220ohm
---> [YELLOW LED] ---> GND | | [5V] --------> | 5V GND |---> Common Ground Rail
+-------------------+ TMP36 : VCC=5V, GND=GND, VOUT=A0 | Vout = (Temp_C + 50) x 10mV LDR : One
end to 5V, other to A1 + 10kohm to GND (voltage divider) PIR : VCC=5V, GND=GND, OUT=D2 | HIGH =
motion detected
```

Pin Connection Table

Component	Arduino Pin	Pin Type	Series Resistor	Function
TMP36 Sensor	A0	Analog IN	None	Temperature voltage output (0–5V)
LDR + 10kΩ	A1	Analog IN	10kΩ pulldown	Light level (ADC 0–1023)
PIR HC-SR501	D2	Digital IN	None	Motion detect HIGH/LOW
Red LED	D9	Digital OUT	220Ω	Temperature alert indicator
Blue LED	D10	Digital OUT	220Ω	Darkness / night indicator

Yellow LED	D11	Digital OUT	220Ω	Motion alert indicator

3. Arduino Sketch (Full Code)

The complete Arduino sketch used in the simulation. It reads all three sensors every 500 ms, applies threshold logic to control the LEDs, and outputs readings to the Serial Monitor.

sketch.ino — Full Arduino Code:

```
// ================================================================ // IoT Sensor-Based LED Control -- Arduino Sketch // Simulation Platform: Tinkercad / Proteus // Sensors: TMP36 (Temp) | LDR (Light) | PIR HC-SR501 (Motion) // ================================================================ // -- Pin Definitions ------------------------------------------- const int TEMP_PIN = A0; // TMP36 analog temperature sensor const int LDR_PIN = A1; // LDR analog light sensor const int PIR_PIN = 2; // PIR digital motion sensor const int LED_TEMP = 9; // Red LED -- temperature alert const int LED_LDR = 10; // Blue LED -- darkness indicator const int LED_PIR = 11; // Yellow LED -- motion alert // -- Threshold Constants ---------------------------------------- const float TEMP_THRESHOLD = 30.0; // Celsius -- trigger above this const int LDR_THRESHOLD = 300; // ADC value -- trigger below this // -- setup() -- runs once on power-on / reset -------------------- void setup() { Serial.begin(9600); // Start serial at 9600 baud pinMode(PIR_PIN, INPUT); // PIR as digital input pinMode(LED_TEMP, OUTPUT); // LED pins as outputs pinMode(LED_LDR, OUTPUT); pinMode(LED_PIR, OUTPUT); Serial.println("IoT System Initialized."); Serial.println("Sensors: TMP36 | LDR | PIR"); } // -- loop() -- runs continuously ------------------------------- void loop() { // 1. READ TEMPERATURE (TMP36 on A0) int rawADC = analogRead(TEMP_PIN); float voltage = rawADC * (5.0 / 1023.0); // convert ADC to volts float tempC = (voltage - 0.5) * 100.0; // TMP36 formula -> Celsius // 2. READ LIGHT LEVEL (LDR on A1) int ldrValue = analogRead(LDR_PIN); // 0 = dark, 1023 = bright // 3. READ PIR MOTION SENSOR (D2) int motionDetected = digitalRead(PIR_PIN); // HIGH = motion present // 4. CONTROL LEDs BASED ON THRESHOLDS digitalWrite(LED_TEMP, tempC > TEMP_THRESHOLD ? HIGH : LOW); digitalWrite(LED_LDR, ldrValue < LDR_THRESHOLD ? HIGH : LOW); digitalWrite(LED_PIR, motionDetected == HIGH ? HIGH : LOW); // 5. SERIAL MONITOR OUTPUT Serial.print("Temp: "); Serial.print(tempC, 1); Serial.print("C | LDR: "); Serial.print(ldrValue); Serial.print(" | Motion: "); Serial.println(motionDetected ? "YES" : "NO"); delay(500); // poll every 500 milliseconds }
```

4. Code Explanation

Section 1 — Pin Definitions

Six constants map meaningful names to Arduino pin numbers. `TEMP_PIN (A0)` and `LDR_PIN (A1)` are analog inputs reading voltage levels from sensors. `PIR_PIN (D2)` is a digital input. LED pins 9, 10, 11 are digital outputs. Using named constants instead of raw numbers makes code readable and easy to modify.

Section 2 — Threshold Constants

`TEMP_THRESHOLD = 30.0` defines the temperature (Celsius) above which the red LED activates — simulating a heat or fire alert. `LDR_THRESHOLD = 300` is the ADC cutoff below which the environment is considered dark, activating the blue night-light LED. These values are easily adjustable for different use cases.

Section 3 — setup() Function

Runs once when the Arduino powers on. `Serial.begin(9600)` initializes communication at 9600 bits per second so the Serial Monitor can display live readings. `pinMode()` calls configure each pin as INPUT or OUTPUT. A startup message confirms the system is ready.

Section 4 — Temperature Reading (TMP36)

`analogRead(TEMP_PIN)` returns a 10-bit ADC value (0–1023). Converted to voltage: `V = ADC × (5.0 / 1023.0)`. Then temperature in Celsius: `Temp = (V − 0.5) × 100`. For example, 750 mV = 25°C. If `tempC > 30.0`, the red LED on pin 9 turns HIGH (on).

Section 5 — Light Sensing (LDR Voltage Divider)

The LDR forms a voltage divider with a 10kΩ resistor. In bright light, LDR resistance drops → higher ADC value. In darkness, resistance rises → lower ADC value. When `ldrValue < 300` (dark environment), the blue LED on pin 10 activates, simulating automatic night lighting.

Section 6 — Motion Detection (PIR HC-SR501)

The PIR sensor outputs a digital HIGH when it detects infrared radiation from a moving warm body. `digitalRead(PIR_PIN)` returns 1 (HIGH) for motion and 0 (LOW) for no motion. The yellow LED on pin 11 mirrors this state directly. The PIR has a built-in hold-time (adjustable via onboard potentiometer) before returning LOW.

Section 7 — Serial Monitor Output

> Every 500 ms, a formatted line prints to the Serial Monitor showing temperature in °C, raw LDR ADC reading, and motion status YES/NO. This enables real-time monitoring and debugging without additional hardware. In Tinkercad, open the Serial Monitor tab to see live output during simulation.

5. Sensor Logic & LED Control Summary

The table below summarizes the complete logic governing each LED in the simulation:

Sensor	Pin	Reading	LED	LED Pin	Trigger Condition
TMP36	A0	Analog 0–1023	RED LED	D9	tempC > 30.0°C → ON
LDR	A1	Analog 0–1023	BLUE LED	D10	ldrValue < 300 → ON
PIR HC-SR501	D2	Digital HIGH/LOW	YELLOW LED	D11	motion == HIGH → ON

How the Thresholds Work

Scenario	Sensor Value	LED State	Real-World Meaning
Room temp = 24°C	ADC ~563	Red OFF	Normal temperature
Temp rises to 32°C	ADC ~737	Red ON	Heat alert triggered
Bright daylight	ADC ~800	Blue OFF	Sufficient ambient light
Dark room / night	ADC ~150	Blue ON	Automatic night light
No movement	LOW (0)	Yellow OFF	Area unoccupied
Person detected	HIGH (1)	Yellow ON	Motion / security alert

6. Sample Serial Monitor Output

The Serial Monitor displays real-time readings every 500 ms. Below is a typical simulation session:

```
IoT System Initialized. Sensors: TMP36 | LDR | PIR
---------------------------------------------- [ 0s] Temp: 24.3C | LDR: 620 | Motion: NO [
1s] Temp: 25.1C | LDR: 580 | Motion: NO [ 2s] Temp: 31.4C | LDR: 570 | Motion: NO << LED_TEMP ON
(D9 HIGH) [ 3s] Temp: 31.4C | LDR: 570 | Motion: YES << LED_PIR ON (D11 HIGH) [ 4s] Temp: 31.4C |
LDR: 250 | Motion: YES << LED_LDR ON (D10 HIGH) [ 5s] Temp: 28.9C | LDR: 250 | Motion: NO <<
LED_TEMP OFF, LED_PIR OFF [ 6s] Temp: 22.0C | LDR: 890 | Motion: NO << LED_LDR OFF (bright again)
```

7. Tinkercad Simulation Steps

Follow these steps to set up and run the simulation in Tinkercad:

Step 1	Open **tinkercad.com**, sign in, and click *Circuits* → *Create new Circuit*.
Step 2	From the components panel, drag an **Arduino UNO** onto the workspace.
Step 3	Add a **TMP36 temperature sensor**. Wire: VCC → 5V, GND → GND, VOUT → A0.
Step 4	Add an **LDR** and a **10kΩ resistor** in a voltage divider. Connect the midpoint to A1; resistor other end to GND; LDR other end to 5V.
Step 5	Add a **PIR HC-SR501** sensor. Wire: VCC → 5V, GND → GND, OUT → D2.
Step 6	Place a **red LED** with a 220Ω resistor in series. Connect anode (through resistor) to D9; cathode to GND.
Step 7	Place a **blue LED** with 220Ω resistor to D10, and a **yellow LED** with 220Ω to D11. All cathodes to GND.
Step 8	Click **Code** in the toolbar → select *Text* mode → paste the full Arduino sketch from Section 3.
Step 9	Click **Start Simulation**. Open the *Serial Monitor* tab to view live sensor readings.
Step 10	Drag the **TMP36 temperature slider** above 30°C to trigger the red LED (heat alert).
Step 11	Cover the LDR by setting its value below 300 to trigger the **blue LED** (night light mode).
Step 12	Click the **PIR sensor** and trigger motion to activate the yellow LED (motion alert).

Expected Simulation Behavior

Action Performed	Expected Result
Slide TMP36 above 30°C	Red LED turns ON immediately
Slide TMP36 below 30°C	Red LED turns OFF
Set LDR to low light (< 300)	Blue LED turns ON
Set LDR to bright light (> 300)	Blue LED turns OFF
Trigger PIR motion	Yellow LED turns ON for hold duration
PIR hold time expires	Yellow LED turns OFF automatically

8. Conclusion

This simulation successfully demonstrates a complete IoT sensor-actuator system using an Arduino UNO. Three sensor types — analog temperature (TMP36), analog light (LDR), and digital motion (PIR) — are integrated into a single sketch applying threshold-based logic to control three LED outputs in real time. The same code deploys to physical hardware without modification, confirming Tinkercad's simulation fidelity.

The project illustrates core IoT engineering concepts including analog-to-digital conversion, digital I/O control, sensor calibration via thresholds, event-driven programming, and serial telemetry for monitoring. The Tinkercad environment accurately replicates real-world Arduino behavior, making it an ideal prototyping platform before physical deployment.

Key Learnings

Concept	Implementation
Analog reading & conversion	TMP36: ADC → Voltage → Celsius formula
Voltage divider circuit	LDR + 10kΩ resistor for light level measurement
Digital I/O control	PIR HIGH/LOW driving LED state directly
Threshold-based logic	Ternary operator `condition ? HIGH : LOW`
Serial communication	9600 baud telemetry for real-time IoT monitoring
Simulation fidelity	Tinkercad matches real-world Arduino behavior exactly
Sensor diversity	Combining analog (A0, A1) and digital (D2) sensors

Possible Extensions

Extension	How to Implement
Buzzer alert for high temp	Add a piezo buzzer on D8; sound when tempC > 35°C
LCD display	Use a 16x2 LCD (I2C) to display sensor values on screen
Fan/relay control	Replace LED with relay module to drive a DC fan above threshold

Wi-Fi IoT upload	Replace Arduino UNO with ESP8266/ESP32 to push data to cloud
Multiple thresholds	Add warning and critical levels with different LED colors

---